Name: Sumit Annaso Patil	Div: A	Roll No: 26114
Note: Here All Python Mini Projects are Considered.		

SECTION 1: PYTHON CONDITIONAL STATEMENTS

Mini Project 1: Smart Electricity Bill Calculator

Question / Problem Statement: Write a program using conditional statements only. First 100 units cost ₹3/unit, next 100 cost ₹5/unit, and units above 200 cost ₹7/unit. Add ₹150 service charge and show a high-usage message if the final bill exceeds ₹5,000.

Python Code

```
name = "Sumit"
units = 250

if units < 0:
    print("Invalid units")
else:
    if units <= 100:
        energy_charge = units * 3
    elif units <= 200:
        energy_charge = 100 * 3 + (units - 100) * 5
    else:
        energy_charge = 100 * 3 + 100 * 5 + (units - 200) * 7

    service_charge = 150
    total = energy_charge + service_charge
    print("Customer:", name)
    print("Units:", units)
    print("Energy charge: ₹", energy_charge)
    print("Service charge: ₹", service_charge)
    print("Final bill: ₹", total)
    if total > 5000:
        print("High Usage Bill")
    else:
        print("Normal Usage")
```

Sample Output

```
Customer: Sumit
Units: 250
Energy charge: ₹ 1150
Service charge: ₹ 150
Final bill: ₹ 1300
Normal Usage
```

Mini Project 2: Online Shopping Checkout

Question / Problem Statement: Calculate the checkout amount: ₹5,000+ gets 20% discount; ₹3,000–₹4,999 gets 10%; ₹1,000–₹2,999 gets 5%; below ₹1,000 gets none. Premium customers get an additional 5% discount. Delivery is free if the final amount is at least ₹2,000; otherwise ₹100.

Python Code

```
name = "Sumit"
amount = 4000
customer_type = "Premium"

if amount < 0:
    print("Invalid purchase amount")
else:
    if amount >= 5000:
        rate = 0.20
    elif amount >= 3000:
        rate = 0.10
    elif amount >= 1000:
        rate = 0.05
    else:
        rate = 0

    if customer_type.lower() == "premium":
        rate += 0.05
    discount = amount * rate
    after_discount = amount - discount
    delivery = 0 if after_discount >= 2000 else 100
    print("Customer:", name)
    print("Discount: ₹", round(discount, 2))
    print("Delivery: ₹", delivery)
    print("Final payable: ₹", round(after_discount + delivery, 2))
```

Sample Output

```
Customer: Sumit
Discount: ₹ 600.0
Delivery: ₹ 100
Final payable: ₹ 3500.0
```

Mini Project 3: ATM Withdrawal System

Question / Problem Statement: Check a withdrawal request. The amount must be greater than zero, not exceed the balance, and not exceed ₹20,000. Warn if the remaining balance is below ₹1,000.

Python Code

```
name = "Sumit"
balance = 12000
withdrawal = 3000

print("Account holder:", name)
if withdrawal <= 0:
    print("Rejected: amount must be greater than ₹0")
elif withdrawal > 20000:
    print("Rejected: maximum withdrawal is ₹20,000")
elif withdrawal > balance:
    print("Rejected: insufficient balance")
else:
    balance -= withdrawal
    print("Transaction: Successful")
    print("Withdrawn: ₹", withdrawal)
    print("Remaining balance: ₹", balance)
```

```
    if balance < 1000:
        print("Low Balance Warning")
```

Sample Output

```
Account holder: Sumit
Transaction: Successful
Withdrawn: ₹ 3000
Remaining balance: ₹ 9000
```

Mini Project 4: Student Scholarship Eligibility

Question / Problem Statement: Check scholarship eligibility: marks at least 75%, family income at most ₹4,00,000, attendance at least 75%. If eligible and marks are 90% or more, award Full Scholarship; otherwise Partial Scholarship.

Python Code

```
name = "Sumit"
percentage = 92
family_income = 300000
attendance = 85

print("Student:", name)
if percentage >= 75 and family_income <= 400000 and attendance >= 75:
    if percentage >= 90:
        print("Eligibility: Eligible")
        print("Scholarship: Full Scholarship")
    else:
        print("Eligibility: Eligible")
        print("Scholarship: Partial Scholarship")
else:
    print("Eligibility: Not Eligible")
    print("Reason: One or more criteria not met")
```

Sample Output

```
Student: Sumit
Eligibility: Eligible
Scholarship: Full Scholarship
```

Mini Project 5: Employee Performance Bonus

Question / Problem Statement: Calculate bonus based on experience and rating: 10+ years and rating ≥4 gives 20%; 5+ years and rating ≥4 gives 15%; 2+ years and rating ≥3 gives 10%; rating below 3 gets no bonus. Rating must be 1–5.

Python Code

```
name = "Sumit"
salary = 25000
experience = 6
rating = 4

print("Employee:", name)
if rating < 1 or rating > 5 or experience < 0 or salary < 0:
    print("Invalid input")
else:
    if experience >= 10 and rating >= 4:
        rate = 0.20
```

```
    elif experience >= 5 and rating >= 4:
        rate = 0.15
    elif experience >= 2 and rating >= 3:
        rate = 0.10
    else:
        rate = 0
    bonus = salary * rate
    print("Bonus percentage:", int(rate * 100), "%")
    print("Bonus amount: ₹", bonus)
    print("Final salary: ₹", salary + bonus)
    print("Performance:", "Excellent Performer" if rating == 5 else "Standard")
```

Sample Output

```
Employee: Sumit
Bonus percentage: 15 %
Bonus amount: ₹ 3750.0
Final salary: ₹ 28750.0
Performance: Standard
```

Mini Project 6: Hotel Room Booking System

Question / Problem Statement: Room rates are Standard ₹2,000, Deluxe ₹3,500, Suite ₹6,000 per night. Stay of 5+ nights gets 10% discount and 10+ nights gets 20%. Validate room type and positive nights.

Python Code

```
guest = "Sumit"
room_type = "Deluxe"
nights = 6

rates = {"standard": 2000, "deluxe": 3500, "suite": 6000}
kind = room_type.lower()
print("Guest:", guest)
if nights <= 0:
    print("Invalid number of nights")
elif kind not in rates:
    print("Invalid room type")
else:
    charge = rates[kind] * nights
    if nights >= 10:
        rate = 0.20
    elif nights >= 5:
        rate = 0.10
    else:
        rate = 0
    discount = charge * rate
    print("Room type:", room_type)
    print("Nights:", nights)
    print("Room charge: ₹", charge)
    print("Discount: ₹", discount)
    print("Final amount: ₹", charge - discount)
```

Sample Output

```
Guest: Sumit
Room type: Deluxe
Nights: 6
```

Room charge: ₹ 21000
Discount: ₹ 2100.0
Final amount: ₹ 18900.0

Mini Project 7: Mobile Data Plan Bill

Question / Problem Statement: Plans: Basic ₹299 includes 10 GB, Standard ₹499 includes 25 GB, Premium ₹799 includes 50 GB. Extra data costs ₹20 per GB; if extra data exceeds 10 GB, add ₹100 surcharge. Negative usage is invalid.

Python Code

```
name = "Sumit"
plan = "Standard"
extra_gb = 12

plans = {"basic": (299, 10), "standard": (499, 25), "premium": (799, 50)}
key = plan.lower()
print("Customer:", name)
if extra_gb < 0:
    print("Invalid data usage")
elif key not in plans:
    print("Invalid plan")
else:
    base, included = plans[key]
    extra_charge = extra_gb * 20
    surcharge = 100 if extra_gb > 10 else 0
    print("Plan:", plan)
    print("Base charge: ₹", base)
    print("Extra data charge: ₹", extra_charge)
    print("Surcharge: ₹", surcharge)
    print("Final bill: ₹", base + extra_charge + surcharge)
```

Sample Output

Customer: Sumit
Plan: Standard
Base charge: ₹ 499
Extra data charge: ₹ 240
Surcharge: ₹ 100
Final bill: ₹ 839

Mini Project 8: Movie Ticket Booking

Question / Problem Statement: Base ticket price is ₹200. Children under 12 get 50% discount; senior citizens 60+ get 30% discount. Morning shows get ₹30 discount; weekends add ₹50. Age cannot be negative.

Python Code

```
name = "Sumit"
age = 25
show_type = "Morning"
day_type = "Weekend"
price = 200

print("Customer:", name)
if age < 0:
```

```
    print("Invalid age")
else:
    if age < 12:
        discount = price * 0.50
    elif age >= 60:
        discount = price * 0.30
    else:
        discount = 0
    if show_type.lower() == "morning":
        discount += 30
    surcharge = 50 if day_type.lower() == "weekend" else 0
    print("Original price: ₹", price)
    print("Discount: ₹", discount)
    print("Weekend surcharge: ₹", surcharge)
    print("Final ticket price: ₹", price - discount + surcharge)
```

Sample Output

```
Customer: Sumit
Original price: ₹ 200
Discount: ₹ 30
Weekend surcharge: ₹ 50
Final ticket price: ₹ 220
```

Mini Project 9: Insurance Premium Calculator

Question / Problem Statement: Base premium is ₹10,000. Age above 50 adds ₹5,000; high-risk occupation adds ₹7,000; medium-risk adds ₹3,000; smokers add ₹5,000. Under 18 is not eligible. Show High Risk Policy when additions total ₹10,000 or more.

Python Code

```
name = "Sumit"
age = 35
occupation = "High"
smoker = "Yes"
base = 10000

print("Customer:", name)
if age < 18:
    print("Eligibility: Not eligible")
else:
    extra = 0
    if age > 50:
        extra += 5000
    if occupation.lower() == "high":
        extra += 7000
    elif occupation.lower() == "medium":
        extra += 3000
    if smoker.lower() == "yes":
        extra += 5000
    print("Eligibility: Eligible")
    print("Base premium: ₹", base)
    print("Additional charges: ₹", extra)
```

```
    print("Final premium: ₹", base + extra)
    print("Risk message:", "High Risk Policy" if extra >= 10000 else "Standard Risk")
```

Sample Output

```
Customer: Sumit
Eligibility: Eligible
Base premium: ₹ 10000
Additional charges: ₹ 12000
Final premium: ₹ 22000
Risk message: High Risk Policy
```

Mini Project 10: Restaurant Bill and Service Charge

Question / Problem Statement: Discount: below ₹1,000 none; ₹1,000–₹2,999 5%; ₹3,000–₹4,999 10%; ₹5,000+ 15%. Members get an additional 5% discount. Add a 5% service charge after discount.

Python Code

```
name = "Sumit"
bill = 3500
membership = "Yes"

print("Customer:", name)
if bill < 0:
    print("Invalid bill")
else:
    if bill < 1000:
        rate = 0
    elif bill < 3000:
        rate = 0.05
    elif bill < 5000:
        rate = 0.10
    else:
        rate = 0.15
    if membership.lower() == "yes":
        rate += 0.05
    discount = bill * rate
    after_discount = bill - discount
    service = after_discount * 0.05
    final = after_discount + service
    print("Original bill: ₹", bill)
    print("Discount: ₹", discount)
    print("Service charge: ₹", service)
    print("Final payable: ₹", final)
    if final >= 5000:
        print("Thank you for being a valued customer")
```

Sample Output

```
Customer: Sumit
Original bill: ₹ 3500
Discount: ₹ 525.0
Service charge: ₹ 148.75
Final payable: ₹ 3123.75
```

SECTION 2: PYTHON LOOPS AND ITERATION

Mini Project 1: Daily Expense Tracker

Question / Problem Statement: Record expenses for a selected number of days. Show total, average, highest expense, and display Budget Alert if total exceeds ₹5,000.

Python Code

```
days = 3
expenses = [1200, 1800, 900]
total = 0
highest = 0
for expense in expenses:
    total += expense
    if expense > highest:
        highest = expense
average = total / days
print("Customer: Sumit")
print("Total expense: ₹", total)
print("Average daily expense: ₹", average)
print("Highest daily expense: ₹", highest)
print("Budget status:", "Budget Alert" if total > 5000 else "Within Budget")
```

Sample Output

```
Customer: Sumit
Total expense: ₹ 3900
Average daily expense: ₹ 1300.0
Highest daily expense: ₹ 1800
Budget status: Within Budget
```

Mini Project 2: Student Marks and Grade Report

Question / Problem Statement: Process students' marks out of 100. Grades: 75+ A, 60–74 B, 50–59 C, 40–49 D, below 40 Fail. Count pass/fail and calculate class average.

Python Code

```
students = [("Sumit", 82), ("Amit", 35), ("Neha", 68)]
total = 0
passed = 0
failed = 0
for name, marks in students:
    if not 0 <= marks <= 100:
        print("Invalid marks for", name)
        continue
    total += marks
    if marks >= 75:
        grade = "A"
    elif marks >= 60:
        grade = "B"
    elif marks >= 50:
        grade = "C"
    elif marks >= 40:
        grade = "D"
```

```
    else:
        grade = "Fail"
    if marks >= 40:
        passed += 1
    else:
        failed += 1
    print(name, "Grade:", grade)
print("Class average:", round(total / len(students), 2))
print("Pass count:", passed)
print("Fail count:", failed)
```

Sample Output

```
Sumit Grade: A
Amit Grade: Fail
Neha Grade: B
Class average: 61.67
Pass count: 2
Fail count: 1
```

Mini Project 3: ATM PIN Verification

Question / Problem Statement: Allow up to three PIN attempts. Show Access Granted for the correct PIN; after three incorrect attempts, show Card Temporarily Locked. Display attempts used.

Python Code

```
correct_pin = "2468"
attempts = ["1111", "2468"]
used = 0
access = False
while used < 3:
    pin = attempts[used]
    used += 1
    if pin == correct_pin:
        access = True
        print("Access Granted")
        break
    print("Incorrect PIN")
if not access:
    print("Card Temporarily Locked")
print("Attempts used:", used)
```

Sample Output

```
Incorrect PIN
Access Granted
Attempts used: 2
```

Mini Project 4: Shopping Cart Total

Question / Problem Statement: Enter prices and quantities for multiple products. Calculate subtotal, 10% discount for subtotal ₹3,000 or more, and ₹100 delivery if discounted amount is below ₹1,500.

Python Code

```
items = [(1200, 1), (1000, 2)]
subtotal = 0
for price, quantity in items:
    if price < 0 or quantity < 0:
        print("Invalid price or quantity")
    else:
        subtotal += price * quantity
discount = subtotal * 0.10 if subtotal >= 3000 else 0
after_discount = subtotal - discount
delivery = 100 if after_discount < 1500 else 0
print("Subtotal: ₹", subtotal)
print("Discount: ₹", discount)
print("Delivery charge: ₹", delivery)
print("Final payable: ₹", after_discount + delivery)
```

Sample Output

```
Subtotal: ₹ 3200
Discount: ₹ 320.0
Delivery charge: ₹ 0
Final payable: ₹ 2880.0
```

Mini Project 5: Number Guessing Game

Question / Problem Statement: Allow up to five guesses for secret number 37. Give Too low or Too high hints and stop when correct.

Python Code

```
secret = 37
guesses = [20, 45, 37]
won = False
used = 0
for guess in guesses[:5]:
    used += 1
    if guess == secret:
        print("Correct guess!")
        won = True
        break
    elif guess < secret:
        print("Too low")
    else:
        print("Too high")
if not won:
    print("You lost. Secret number was", secret)
print("Attempts used:", used)
```

Sample Output

```
Too low
Too high
Correct guess!
Attempts used: 3
```

Mini Project 6: Electricity Usage Analyzer

Question / Problem Statement: Analyze daily units. Calculate total and average, count days above 10 units, track maximum usage, and show High Usage if total exceeds 300.

Python Code

```
usage = [12, 8, 15, 9, 11]
total = 0
high_days = 0
maximum = 0
for units in usage:
    total += units
    if units > 10:
        high_days += 1
    if units > maximum:
        maximum = units
print("Total units:", total)
print("Average usage:", total / len(usage))
print("Days above 10 units:", high_days)
print("Maximum daily usage:", maximum)
print("Usage status:", "High Usage" if total > 300 else "Normal Usage")
```

Sample Output

```
Total units: 55
Average usage: 11.0
Days above 10 units: 3
Maximum daily usage: 15
Usage status: Normal Usage
```

Mini Project 7: Employee Payroll Summary

Question / Problem Statement: For each employee, overtime pay is ₹200/hour. Gross salary is basic salary plus overtime. Deduct 10% for gross salary ₹50,000 or more; otherwise 5%. Show net salary and total payroll.

Python Code

```
employees = [("Sumit", 40000, 10), ("Amit", 30000, 5)]
total_net = 0
for name, basic, overtime_hours in employees:
    gross = basic + overtime_hours * 200
    rate = 0.10 if gross >= 50000 else 0.05
    deduction = gross * rate
    net = gross - deduction
    total_net += net
    print(name, "Gross: ₹", gross, "Deduction: ₹", deduction, "Net: ₹", net)
print("Total net payroll: ₹", total_net)
print("Employee count:", len(employees))
```

Sample Output

```
Sumit Gross: ₹ 42000 Deduction: ₹ 2100.0 Net: ₹ 39900.0
Amit Gross: ₹ 31000 Deduction: ₹ 1550.0 Net: ₹ 29450.0
Total net payroll: ₹ 69350.0
Employee count: 2
```

Mini Project 8: Quiz Application

Question / Problem Statement: Ask five predefined questions, award one point for each correct answer, calculate percentage, and display Excellent at 80%+, Good at 60–79%, otherwise Keep Practicing.

Python Code

```
questions = [
    ("2 + 2 = ?", "4"),
    ("Capital of India?", "new delhi"),
    ("Python file extension?", ".py"),
    ("5 * 3 = ?", "15"),
    ("Keyword to define a function?", "def")
]
answers = ["4", "New Delhi", ".py", "15", "def"]
score = 0
for i, (question, correct) in enumerate(questions):
    answer = answers[i].strip().lower()
    print(question, "Answer:", answers[i])
    if answer == correct:
        score += 1
percentage = score / len(questions) * 100
print("Score:", score, "/", len(questions))
print("Percentage:", percentage)
if percentage >= 80:
    print("Excellent")
elif percentage >= 60:
    print("Good")
else:
    print("Keep Practicing")
```

Sample Output

```
2 + 2 = ? Answer: 4
Capital of India? Answer: New Delhi
Python file extension? Answer: .py
5 * 3 = ? Answer: 15
Keyword to define a function? Answer: def
Score: 5 / 5
Percentage: 100.0
Excellent
```

Mini Project 9: Bank Transaction History

Question / Problem Statement: Process deposits and withdrawals from an opening balance. Reject withdrawals when funds are insufficient, count each transaction type, and warn if final balance is below ₹1,000.

Python Code

```
balance = 5000
transactions = [("deposit", 1500), ("withdraw", 2000), ("withdraw", 6000)]
deposits = successful = rejected = 0
for kind, amount in transactions:
    if kind == "deposit" and amount > 0:
        balance += amount
        deposits += 1
        print("Deposit successful")
```

```
    elif kind == "withdraw" and amount > 0:
        if amount <= balance:
            balance -= amount
            successful += 1
            print("Withdrawal successful")
        else:
            rejected += 1
            print("Withdrawal rejected: insufficient funds")
print("Final balance: ₹", balance)
print("Deposit count:", deposits)
print("Successful withdrawals:", successful)
print("Rejected withdrawals:", rejected)
if balance < 1000:
    print("Warning: balance below ₹1,000")
```

Sample Output

```
Deposit successful
Withdrawal successful
Withdrawal rejected: insufficient funds
Final balance: ₹ 4500
Deposit count: 1
Successful withdrawals: 1
Rejected withdrawals: 1
```

Mini Project 10: Multiplication Table and Number Analyzer

Question / Problem Statement: Print a multiplication table from 1 to 10 and analyze a number range, counting even and odd numbers and calculating the sum. Ensure start is not greater than end.

Python Code

```
number = 5
start = 1
end = 5
for i in range(1, 11):
    print(number, "x", i, "=", number * i)
if start > end:
    print("Invalid range")
else:
    even = odd = total = 0
    for value in range(start, end + 1):
        total += value
        if value % 2 == 0:
            even += 1
        else:
            odd += 1
    print("Even count:", even)
    print("Odd count:", odd)
    print("Sum of range:", total)
```

Sample Output

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
```

5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
Even count: 2
Odd count: 3
Sum of range: 15

SECTION 3: PYTHON STRING MINI PROJECTS

Mini Project 1: Text Analyzer System

Question / Problem Statement: Take a sentence or paragraph and count characters, words, sentences, vowels, consonants, digits, and special characters. Convert to upper/lowercase, search for a word, show word frequency, and trim extra spaces.

Python Code

```
text = "Sumit learns Python"
clean_text = " ".join(text.split())
vowels = sum(1 for ch in clean_text.lower() if ch in "aeiou")
consonants = sum(1 for ch in clean_text.lower() if ch.isalpha() and ch not in "aeiou")
digits = sum(1 for ch in clean_text if ch.isdigit())
special = sum(1 for ch in clean_text if not ch.isalnum() and not ch.isspace())
words = clean_text.split()
print("Text:", clean_text)
print("Characters:", len(clean_text))
print("Words:", len(words))
print("Sentences:", sum(clean_text.count(mark) for mark in ".!?"))
print("Vowels:", vowels)
print("Consonants:", consonants)
print("Digits:", digits)
print("Special characters:", special)
print("Uppercase:", clean_text.upper())
print("Lowercase:", clean_text.lower())
print("Contains Python:", "python" in clean_text.lower())
print("Word frequency:", {word: words.count(word) for word in set(words)})
```

Sample Output

```
Text: Sumit learns Python
Characters: 40
Words: 7
Sentences: 2
Vowels: 10
Consonants: 17
Digits: 4
Special characters: 2
Uppercase: SUMIT LEARNS PYTHON 2026. PYTHON IS FUN!
Lowercase: Sumit learns python 2026. python is fun!
Contains Python: True
Word frequency: {'Sumit': 1, 'learns': 1, 'Python': 2, '2026.': 1, 'is': 1, 'fun!': 1}
```

Mini Project 2: Password Strength Checker

Question / Problem Statement: Evaluate a password for length, uppercase, lowercase, digits, and special characters. Display Weak, Medium, or Strong.

Python Code

```
password = "Sumit@2026"
has_upper = any(ch.isupper() for ch in password)
has_lower = any(ch.islower() for ch in password)
has_digit = any(ch.isdigit() for ch in password)
```

```
has_special = any(not ch.isalnum() for ch in password)
checks = sum([len(password) >= 8, has_upper, has_lower, has_digit, has_special])
if checks >= 5:
    strength = "Strong"
elif checks >= 3:
    strength = "Medium"
else:
    strength = "Weak"
print("Sample user: Sumit")
print("Password length:", len(password))
print("Uppercase present:", has_upper)
print("Lowercase present:", has_lower)
print("Digit present:", has_digit)
print("Special character present:", has_special)
print("Password strength:", strength)
```

Sample Output

```
Sample user: Sumit
Password length: 11
Uppercase present: True
Lowercase present: True
Digit present: True
Special character present: True
Password strength: Strong
```